Agentic Coding with OpenCode

Q-Summit 2026

Building software with AI agents that plan, act, and observe

`50-minute workshop`

Agenda

Time	Section
00:00–05:00	What is Agentic Coding?
05:00–15:00	OpenCode: Architecture & Core Concepts
15:00–25:00	MCP Servers & Permissions
25:00–40:00	Live Demos (Obsidian Plugin + CLI)
40:00–50:00	Patterns, Pitfalls & Q&A

Goal: Leave able to run OpenCode on a real project today.

What is Agentic Coding?

Traditional coding assistants → **autocomplete / chat**

Agentic coding → **plan** → **act** → **observe** → **repeat**

The key differences:

- **Autonomous multi-step execution** — the agent decides the next action
- **Tool use** — reads files, runs bash, searches the web
- **Feedback loops** — observes output and self-corrects
- **Configurable trust boundaries** — you decide what's auto-approved

> "You are now a junior developer on your team — except it ships code instantly"

OpenCode — What It Is

OpenCode is an open-source AI coding agent built for the terminal.

```
# Install
curl -fsSL https://opencode.ai/install | bash
# or
npm install -g opencode-ai

# Run in your project
cd my-project && opencode
```

- Open-source (GitHub: anomalyco/opencode)
- Works with any LLM provider (Anthropic, OpenAI, Bedrock, local models)
- TUI, desktop app, IDE extension, CLI — same config everywhere
- First-class multi-agent orchestration built in

OpenCode Architecture

```
┌──────────────────────────────────────┐
│            User Interface            │
│  TUI  │  IDE Extension  │  Web       │
└──────────────────────────────────────┘
                │
                ▼
┌──────────────────────────────────────┐
│          OpenCode Core               │
│  Agent Loop  │  Tool Router          │
│  Permissions │  Context Manager      │
└──────────────────────────────────────┘
        │                  │
        ▼                  ▼
┌──────────┐        ┌──────────────┐
│  LLM     │        │    Tools     │
│Provider  │        │bash│read│MCP  │
└──────────┘        └──────────────┘
```

Config files: `opencode.json` (global or per-project) + `AGENTS.md`

The Agent Loop

```
User Message
     │
     ▼
┌──────────────┐    tool call    ┌──────────────┐
│  LLM Plan    │ ──────────────► │ Tool Execute │
│  (think)     │                 │  (act)       │
└──────────────┘ ◄────────────── └──────────────┘
                    tool result
     │                                  │
     ▼                                  ▼
 Next step?                      Permission check
 Done? Ask?                      allow / ask / deny
```

Modes:

- **Build** (default) — full tool access
- **Plan** — read-only, edits require approval → switch with `Tab`

Built-in Tools

Tool	Purpose
`bash`	Run shell commands (`npm install` , `git status`)
`read`	Read file contents (line-range aware)
`edit`	Exact string replacement in files
`write`	Create or overwrite files
`glob`	Find files by pattern (`**/*.ts`)
`grep`	Regex search across codebase (uses ripgrep)
`list`	List directory contents

MCP Servers — What & Why

MCP = Model Context Protocol

A standard for connecting LLMs to external tools and services.

Why it matters:

- Extend OpenCode with **any external API** (Sentry, Jira, GitHub, databases)
- Tools become first-class citizens alongside built-ins
- The LLM uses them the same way — just another tool call
- Context cost: each MCP server adds tokens → use selectively

```
OpenCode ◀──── stdio (local) ────▶ MCP Process
OpenCode ◀──── HTTPS (remote) ────▶ MCP Server
```

MCP Config: Local Server

```
{
  "$schema": "https://opencode.ai/config.json",
  "mcp": {
    "filesystem": {
      "type": "local",
      "command": ["npx", "-y", "@modelcontextprotocol/server-
                  "/path/to/allowed"],
      "enabled": true,
      "environment": {
        "DEBUG": "1"
      }
    }
  }
}
```

MCP Config: Remote + OAuth

```
{
  "$schema": "https://opencode.ai/config.json",
  "mcp": {
    "sentry": {
      "type": "remote",
      "url": "https://mcp.sentry.dev/mcp",
      "oauth": {}
    },
    "context7": {
      "type": "remote",
      "url": "https://mcp.context7.com/mcp",
      "headers": {
        "CONTEXT7_API_KEY": "{env:CONTEXT7_API_KEY}"
      }
    }
  }
}
```

Permissions — Why They Matter

Without permissions: the agent can do anything.

- Delete production config
- Push to main without review
- Read `.env` files with secrets
- Execute `rm -rf` variants

OpenCode permission model: 3 outcomes per tool:

- `"allow"` — run without prompt
- `"ask"` — show UI dialog (once / always / reject)
- `"deny"` — block entirely

Default: most tools `allow`, `.env` files `deny`, `doom_loop` and `external_directory` → `ask`

Permission Config — Real Examples

```json
{
  "$schema": "https://opencode.ai/config.json",
  "permission": {
    "*": "ask",
    "bash": {
      "*": "ask",
      "git *": "allow",
      "npm *": "allow",
      "rm *": "deny",
      "grep *": "allow"
    },
    "edit": {
      "*": "deny",
      "packages/web/src/content/docs/*.mdx": "allow"
    },
    "read": {
      "*": "allow",
      "*.env": "deny",
      "*.env.*": "deny",
      "*.env.example": "allow"
    }
  }
}
```

AGENTS.md — Project Rules

Two types of rule files:

File	Scope	Purpose
`AGENTS.md` (project root)	This repo	Architecture, commands, conventions
`~/.config/opencode/AGENTS.md`	All sessions	Personal preferences

Initialized via `/init` — OpenCode scans your repo and generates it.

```
# My Project

## Build Commands
- `npm run build` — full build
- `npm test` — test suite

## Conventions
- Use TypeScript strict mode
- All API calls go through `src/api/client.ts`
```

Agent Files — Configuration Shape

Markdown agents live in:

- Global: `~/.config/opencode/agents/review.md`
- Project: `.opencode/agents/review.md`

```
---
description: Code review without making edits
mode: subagent
model: anthropic/claude-sonnet-4-20250514
temperature: 0.1
permission:
  edit: deny
  bash:
    "*": ask
    "git diff": allow
    "git log*": allow
    "grep *": allow
  webfetch: deny
---

You are a code reviewer. Focus on:
- Security and input validation
- Performance implications
- Code clarity and maintainability
```

Agent Config: JSON Style

```
{
  "$schema": "https://opencode.ai/config.json",
  "agent": {
    "build": {
      "mode": "primary",
      "model": "anthropic/claude-sonnet-4-20250514",
      "permission": {
        "bash": {
          "*": "ask",
          "git status *": "allow",
          "npm run *": "allow"
        }
      }
    },
    "code-reviewer": {
      "description": "Reviews code for best practices",
      "mode": "subagent",
      "permission": {
        "edit": "deny",
        "bash": "deny"
      }
    }
  }
}
```

Built-in Agents

Primary (cycle with Tab):

- `build` — full access (default)
- `plan` — edits/bash require ask

Subagents (@ mention or auto):

- `general` — full access, parallel tasks
- `explore` — read-only, fast codebase search

Hidden system agents:

- `compaction` — auto-compacts long contexts
- `title` — generates session titles
- `summary` — session summaries

Skills — Reusable Instruction Sets

Skills are named `SKILL.md` files loaded on-demand by agents.

```
.opencode/skills/git-release/SKILL.md
~/.config/opencode/skills/deploy-checklist/SKILL.md
```

```
---
name: git-release
description: Create consistent releases and changelogs
license: MIT
compatibility: opencode
---

## What I do
- Draft release notes from merged PRs
- Propose a version bump
- Provide a copy-pasteable `gh release create` command

## When to use me
Use when preparing a tagged release.
```

Agent sees available skills in tool description → loads on demand via `skill({ name: "git-release" })`

Demo 1: Obsidian Plugin Scaffold

Goal: Use OpenCode to scaffold a new Obsidian plugin from scratch.

Setup:

```
cd workshop-agentic/demo-repo/obsidian-plugin
opencode
```

Steps we'll follow:

1. Switch to **Plan mode** (Tab) — ask for architecture overview
2. Review the plan together
3. Switch to **Build mode** (Tab) — execute
4. Watch agent: read existing examples → write plugin boilerplate → run `npm install`
5. Use `/undo` to revert one change and re-prompt

What to observe:

- Tool calls streamed live (bash, write, read)

Demo 1: Walkthrough Notes

```
Plan output example:
1. Create manifest.json with plugin metadata
2. Create main.ts with Plugin class extending Plugin
3. Create styles.css for status bar styling
4. Create package.json with esbuild config
5. Run npm install

Build execution:
[write] manifest.json          → auto-allow (new file)
[write] main.ts                → auto-allow (new file)
[bash] npm install             → PERMISSION PROMPT
  > once | always | reject
[bash] npm run build           → PERMISSION PROMPT
```

Key observations:

- Agent reads `obsidian` npm package types to understand the API
- Glob searches for existing examples before writing
- Context7 MCP could provide live Obsidian API docs

Demo 2: CLI Plan Mode on Existing Repo

Goal: Analyze a messy CLI repo, get a refactoring plan — no changes made.

```
cd workshop-agentic/demo-repo/cli-fallback
opencode run --agent plan "Analyze this CLI codebase.
Identify the top 3 refactoring opportunities.
Output a structured plan with effort estimates."
```

This uses:

- `opencode run` — single-shot, non-interactive
- `--agent plan` — forces Plan mode (no edits)
- Stdout output suitable for CI pipelines

Demo 2: Walkthrough

```
# Non-interactive analysis
opencode run --agent plan \
  "Find all TODO comments and create a prioritized issue list"

# Pipe output to a file
opencode run --agent plan \
  "Summarize the public API surface of this module" > api-summary.md

# With custom AGENTS.md context
OPENCODE_CONFIG=./opencode.ci.json opencode run \
  "Check for any hardcoded credentials or API keys"
```

Real-world CI pattern:

```
# .github/workflows/ai-review.yml
- name: AI Code Review
  run: |
    opencode run --agent plan \
      "Review this PR diff for security issues. Be concise."
```

Real-World Patterns

Pattern 1: Trust Tiers

```
read-only tier:    plan agent, explore subagent
restricted tier:   git allow, npm allow, rm deny
full-trust tier:   only in dev, never in CI
```

Pattern 2: Per-Agent MCP Scoping

```
{ "tools": { "my-mcp*": false },
  "agent": { "research": { "tools": { "my-mcp*": true } } } }
```

→ MCP tools only available to the agent that needs them

Pattern 3: Skills for Team Workflows

```
.opencode/skills/release-checklist/SKILL.md
.opencode/skills/pr-template/SKILL.md
.opencode/skills/security-review/SKILL.md
```

→ Shared, versioned, Git-committed team knowledge

Pattern 4: AGENTS.md as Living Docs Update after every significant architecture decision.

Pitfalls & Anti-Patterns

❌ "Allow everything" in shared environments

```
{ "permission": "allow" }  // NEVER in CI or shared machines
```

❌ **Secrets in config files**

```
{ "headers": { "Authorization": "Bearer sk-abc123" } }  // bad
{ "headers": { "Authorization": "Bearer {env:MY_KEY}" } } // good
```

❌ **Ignoring `doom_loop` warnings** Three identical tool calls = agent is stuck. Intervene, don't ignore.

❌ **Giant AGENTS.md** The whole file is injected every session. Keep it focused, use Skills for detail.

❌ **No `/init` before starting** Without AGENTS.md, the agent guesses your conventions. Run `/init` first.

✅ **Always start with Plan mode for unfamiliar codebases**

Q&A — Discussion Prompts

For the audience:

1. Where in your current workflow would an agentic step add the most value?
2. What's the riskiest thing you'd worry about giving an agent access to?
3. Has anyone tried MCP servers beyond the built-ins? What worked?

Resources:

- Docs: https://opencode.ai/docs
- GitHub: https://github.com/anomalyco/opencode
- Discord: https://opencode.ai/discord
- This workshop: `workshop-agentic/` in the Q-Summit repo

Thank You

Workshop materials:

```
git clone <q-summit-repo>
cd workshop-agentic
# Follow README.md
```

OpenCode: `curl -fsSL https://opencode.ai/install | bash`

Questions, feedback, follow-up: use the Q-Summit Discord channel **#agentic-coding**